Unified Marketplace Architecture

An in-depth explanation of the proposed single Marketplace UI across Dev, Stage, and Prod

Core idea: Provide one Marketplace frontend experience while continuing to use separate environment-specific backends and agent deployments. Marketplace becomes environment-aware; it does not necessarily merge the environments themselves.

Interpretation note: This document is based on the meeting transcript. Where the discussion did not specify an implementation detail, it is identified as a proposal, example, or open decision.

1. What problem are they trying to solve?

The discussion suggests that Marketplace is currently organized around separate Dev, Stage, and Prod experiences or configurations. This can mean repeated setup, different URLs, and extra coordination when users move an agent from development toward production.

DEV Marketplace	STAGE Marketplace	PROD Marketplace
Dev backend / UACP	Stage backend / UACP	Prod backend / UACP
Dev agents	Stage agents	Prod agents

The team wants to avoid treating an agent as something users must independently register and manage through a different Marketplace experience for every environment. Instead, one Marketplace should provide the interface for discovering agents, selecting the appropriate environment, and accessing supported functions.

2. What is the proposed architecture?

ONE UNIFIED MARKETPLACE UI

↓ Environment selection / context and routing decision ↓

DEV backend / UACP	STAGE backend / UACP	PROD backend / UACP

The user sees one Marketplace, but the system still communicates with the appropriate environment-specific services. Selecting Stage should not accidentally invoke Dev or Prod. The environment determines the correct destination and, potentially, which actions are available.

Important distinction: A unified UI is not the same thing as a unified runtime. Dev, Stage, and Prod can remain independently deployed and isolated.

3. Walkthrough: using the Playground

Imagine a developer opens the Claims Assistant agent inside the unified Marketplace. The Playground has an environment selector with Dev and Stage available for testing.

Example A - Dev: The user chooses Dev and sends a message. Marketplace sends the request to the Dev UACP/backend, which invokes the Dev deployment of the selected agent. The response comes back to the same Playground UI.

Example B - Stage: The user switches the environment selector to Stage. Marketplace must now target the Stage UACP/backend and the Stage deployment of that agent, not the Dev deployment.

Example C - Prod: The user can see the production context where appropriate, but the team specifically said production testing from the Marketplace Playground should be disabled. This is a UI restriction and should also be enforced by the relevant backend authorization/policy.

4. Why environment-to-backend mapping matters

The meeting repeatedly mentioned mapping environments to URLs. A selection in the UI must resolve to the correct downstream service. Conceptually, the mapping could look like this:

Selected environment	Target service	Playground behavior
Dev	Dev UACP / backend URL	Testing enabled
Stage	Stage UACP / backend URL	Testing enabled
Prod	Prod UACP / backend URL	Testing disabled

Two possible designs are: (1) the frontend resolves an environment-specific backend URL; or (2) the frontend calls a common API and passes the environment, while a gateway/backend performs the routing. The transcript does not establish which approach was selected.

5. What does agent promotion mean here?

The desired user experience is that an agent moves through Dev, Stage, and Prod through promotion, instead of being manually registered in separate Marketplaces each time. The underlying promotion process may still create or update environment-specific deployments.

Agent in Dev → Promote to Stage → Promote to Prod

This is part of the overall product vision, but it may be owned by a separate agent-management team. Marketplace may only need to display the environment status and provide an entry point to promotion-related actions or APIs.

6. What changes inside Marketplace?

• **Environment context:** Decide where the user selects an environment and whether it applies globally or only to specific features.

• **Environment-aware API calls:** Route agent lookup, testing, and other supported operations to the correct environment.

• **Playground behavior:** Enable testing in allowed environments and prevent it in Prod.

• **Agent visibility:** Determine whether Marketplace shows one logical agent with per-environment availability or separate entries.

• **Configuration:** Manage environment-specific URLs and feature flags without unnecessary duplication.

• **Cross-team contracts:** Understand what the agent-management service returns about agent identity, availability, and promotion status.

7. What likely belongs to your team versus another team?

Marketplace / UI team (likely)	Agent management team (likely)
Environment selector and user experience	Actual agent registration and lifecycle
Playground integration and environment-specific routing	Promotion mechanics and deployment changes
Production testing restrictions in Marketplace	Agent versioning, promotion validation, and rollback
Displaying environment availability and status	Maintaining authoritative agent deployment state

These ownership boundaries are inferred from your description, not confirmed assignments.

8. Architectural decisions the team still needs to clarify

• **What is shared?** Only the UI, or also a common API/gateway and shared agent catalog?

• **Who owns routing?** Does Marketplace resolve backend URLs, or does a common backend route requests?

• **What is environment context?** Is it a global Marketplace selection or specific to each workflow?

• **What identifies an agent?** Is there a shared logical ID with separate environment-specific deployment IDs?

• **How is Prod testing blocked?** What is enforced in the UI, and what must be rejected server-side?

• **What data comes from agent management?** How does Marketplace learn where an agent exists and what actions are allowed?

9. A realistic end-to-end example

Scenario: An engineer wants to test Claims Assistant in Stage.

1. The engineer opens the unified Marketplace and selects Claims Assistant.

2. They choose Stage in the Playground.

3. Marketplace determines whether the agent is available in Stage and whether testing is permitted.

4. Marketplace or a routing layer resolves the Stage UACP/backend.

5. The request is sent to the Stage deployment, and the response is displayed in the Playground.

6. If they switch to Prod, the Marketplace does not permit Playground testing.

10. The simplest way to explain it in the meeting

“We are consolidating the Marketplace frontend, not necessarily the backend environments. Users should have one place to work with agents across Dev, Stage, and Prod. Marketplace needs to understand which environment a user is operating in, route requests to the correct UACP/backend, and enforce environment-specific behavior - especially disabling Playground testing in Prod. The agent-management team may own promotion itself; our side needs to integrate with that workflow.”

Bottom line: The main Marketplace engineering challenge is making one UI safely and consistently operate across multiple isolated environments.